

1. Single Responsibility Principle

Related modules	Description	Improvement Direction
Class BaseController	Lớp BaseController không hoàn toàn là "base" theo nghĩa thuần túy của một lớp cha dùng chung. Nó chứa logic cụ thể liên quan đến giỏ hàng, không phù hợp nếu có các controller khác không liên quan đến giỏ hàng.	Tách các phương thức liên quan đến giỏ hàng (e.g., checkMediaInCart, getListCartMedia) vào một lớp tiện ích (e.g., CartService). Giữ BaseController thuần túy như một lớp cha để các controller khác kế thừa nếu cần chia sẻ chung các chức năng như logging hoặc xử lý request cơ bản.
Class PlaceOrderController	Lớp đảm nhận quá nhiều trách nhiệm, vi phạm nguyên lý SRP: • Kiểm tra tính khả dụng của sản phẩm. • Tạo đơn hàng (createOrder) và hóa đơn (createInvoice). • Xử lý và xác thực thông tin giao hàng. • Tính phí vận chuyển.	Tạo các lớp hoặc dịch vụ riêng như: • OrderService để quản lý logic đặt hàng. • InvoiceService để quản lý logic hóa đơn. • ShippingService để xử lý tính phí vận chuyển. • DeliveryInfoValidator để xác thực thông tin giao hàng. Controller: Chỉ nên điều phối luồng xử lý và gọi các dịch vụ trên.

2. Open-Closed Principle

Related modules	Description	Improvement Direction
ViewCartController	ViewCartController phụ thuộc trực tiếp vào lớp Cart qua các phương thức tĩnh (getCart).	Tách biệt lớp Cart khỏi BaseController bằng cách sử dụng interface
DBConnection	DBConnection chỉ hỗ trợ SQLite với cấu hình cố định. Nếu cần hỗ trợ nhiều loại cơ sở dữ liệu (MySQL, PostgreSQL), phải thay đổi logic trong lớp này.	Trừu tượng hóa logic kết nối cơ sở dữ liệu bằng cách sử dụng một interface

Related modules	Description	Improvement Direction
	Chuỗi kết nối (connectionString) được viết cứng trong mã, không dễ cấu hình.	
Media, Book, CD, DVD	Phương thức getMediaById và getAllMedia chứa logic truy vấn SQL trực tiếp trong lớp, khiến việc mở rộng cách thức truy vấn hoặc thay đổi cấu trúc dữ liệu khó khăn.	Trích xuất logic truy vấn cơ sở dữ liệu ra một lớp DAO (Data Access Object)
Order	Phương thức getAmount có logic tính toán trực tiếp trong lớp. Nếu cần thay đổi cách tính (ví dụ: thêm chiết khấu, phí dịch vụ), phải sửa đổi mã trong lớp này. Dữ liệu giao hàng (deliveryInfo) sử dụng HashMap, khiến việc mở rộng thêm các trường thông tin cụ thể (vd: số điện thoại, mã bưu chính) khó khăn.	Tách logic tính toán tổng số tiền (getAmount) sang một lớp service riêng để dễ mở rộng Thay thế HashMap trong deliveryInfo bằng một lớp cụ thể
PaymentTransaction	Logic truy vấn SQL nằm trực tiếp trong lớp, khiến việc thay đổi hoặc mở rộng cách lưu trữ dữ liệu (vd: chuyển từ SQL sang NoSQL) phải sửa đổi mã trong lớp này. Phương thức save sử dụng câu truy vấn SQL cứng, và một lỗi xảy ra trong truy vấn có thể làm ảnh hưởng đến lớp.	Trích xuất logic truy vấn dữ liệu ra một lớp DAO (Data Access Object) riêng
API	Chưa hoàn toàn tuân thủ , vì logic xử lý HTTP lặp lại trong các phương thức get và post.	Tạo một phương thức riêng để thiết lập và gửi yêu cầu HTTP, dùng chung cho cả get và post
Config	Lớp Configs không tách biệt các loại cấu hình	Tách lớp Configs thành các nhóm cấu hình cụ thể hơn

Related modules	Description	Improvement Direction
Utils	Lớp chứa các phương thức tiện ích hỗn hợp mà không được tổ chức theo nhóm chức năng rõ ràng (vd: xử lý chuỗi, ngày giờ, mã hóa). Nếu cần thêm nhiều loại tiện ích mới, lớp có thể trở nên quá tải và vi phạm OCP.	Tách các tiện ích thành các lớp nhỏ hơn, chuyên biệt hơn
SplashForm	Đường dẫn hình ảnh logo (src/main/resources/...) được mã hóa cứng trong phương thức initialize. Nếu cần thay đổi vị trí hoặc quản lý hình ảnh khác, phải chỉnh sửa trực tiếp lớp, vi phạm OCP.	Trích xuất đường dẫn hình ảnh thành một cấu hình hoặc tham số để dễ dàng mở rộng mà không cần thay đổi mã

3. Liskov Substitution Principle

Related modules	Description	Improvement Direction
PaymentTransaction	Nếu trong tương lai, PaymentTransaction được mở rộng (ví dụ: thêm các loại giao dịch khác nhau như RefundTransaction), cần đảm bảo các lớp con giữ nguyên hành vi của các phương thức như isSuccess.	Sử dụng abstraction để định nghĩa các hành vi chung

4. Interface Segregation Principle

Related modules	Description	Improvement Direction
Thêm interface cho quản lý order	Một đơn hàng có 3 quy trình: xử lý thông tin giao hàng, tạo đơn hàng và tính toán phí vận chuyển	Chia thành 3 Interface với các method: 1. IDeliveryInfoProcessor (validateDeliveryInfo, processDeliveryInfo), 2. IOrderCreator (createOrder, createInvoice), 3. IShippingCalculator (calculateShippingFee)
Triển khai Factory design pattern cho việc lựa chọn cổng thanh toán	Trong tương lai, chúng ta có thể sẽ phải tích hợp nhiều phương thức thanh toán khác nhau.	1. IPaymentMethodFactory (Nơi chọn phương thức thanh toán), 2. IPaymentMethod (Khi tích hợp phương thức thanh toán mới, chỉ cần triển khai

		interface này và thực hiện logic thanh toán bằng cách ghi đề phương thức)
--	--	---

5. Dependency Inversion Principle

Related modules	Description	Improvement Direction
Media	Constructor Media gọi đến phương thức getConnection của lớp DBConnection. Trong tương lai nếu ta muốn sử dụng cơ sở dữ liệu khác, ta sẽ phải chỉnh sửa code	Tạo một interface IDatabase và làm cho lớp DBConnection implement nó. Trong constructor Media, ta chỉ cần phải gọi phương thức getConnection của interface IDatabase và ta có thể xác định sử dụng cơ sở dữ liệu nào ở constructor
Book	Method getMediaById call DBConnection.getConnection. In the future if we want to use another DB, we must modify the code Phương thức getMediaById gọi đến phương thức getConnection của lớp DBConnection. Trong tương lai nếu ta muốn sử dụng cơ sở dữ liệu khác, ta sẽ phải chỉnh sửa code	Tạo một interface IDatabase và làm cho lớp DBConnection implement nó. Trong phương thức getMediaById, ta chỉ cần phải gọi phương thức getConnection của interface IDatabase và ta có thể xác định sử dụng cơ sở dữ liệu nào ở constructor